Trekking Management Application V2
Project Report

1. Student Details

Name: Vishesh Sahu

Roll Number: 23F3001373

Email: 23f3001373@ds.study.iitm.ac.in

About Me: I am a student of the IITM BS Degree in Data Science and Applications.

2. Project Details

Project Title: Trekking Management Application V2

Problem Statement: Designing and building a multi-role, responsive web app to streamline trekking route operations, guide allocations, and bookings. The app must manage slots programmatically without race conditions, support caching on search endpoints, facilitate background scheduling (daily emails, monthly logs, asynchronous CSV exports), and work fully offline as a PWA.

Approach: The architecture consists of a Flask REST API backend storing data in SQLite via SQLAlchemy ORM, and a responsive frontend Vue.js Single Page Application styled with Bootstrap 5. Redis is utilized for endpoint caching, and Celery serves background tasks. Database locks protect booking operations to prevent overbooking, and active cache invalidation hooks keep counts synchronized.

3. AI/LLM Declaration

During the development, LLM (Google's Gemini) were utilized strictly as a supplementary learning tool, debugging assistant, and syntax reference. The AI was used to accelerate the learning process and overcome technical roadblocks.

The extent of AI/LLM usage is around 20%, limited to assist with syntax and troubleshooting. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

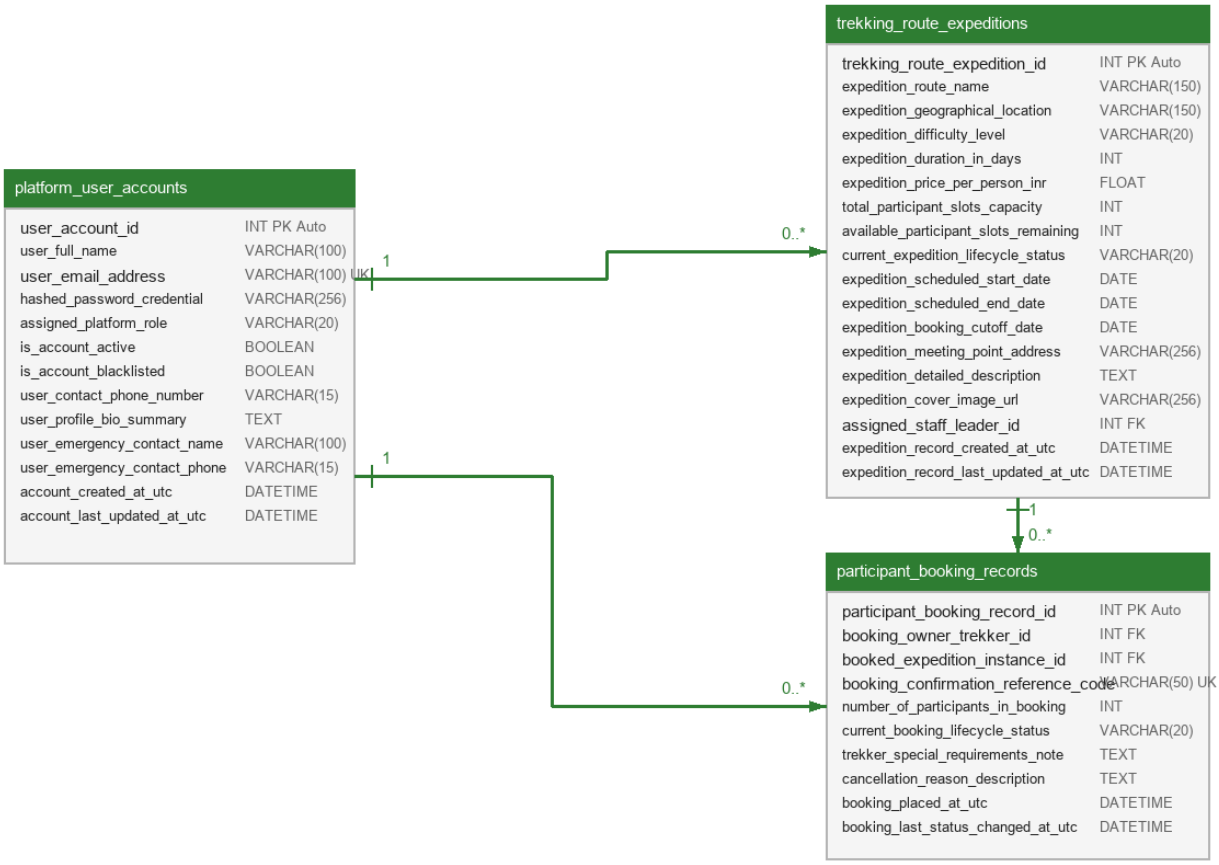
Technology / Library	Purpose
Flask	Core backend REST API web framework
SQLAlchemy ORM	Database abstraction and programmatic transactional models
Vue.js 3	Frontend dynamic single-page app (SPA) controller
Bootstrap 5	Core responsive design system and glassmorphism layout
Redis	In-memory caching and message queue broker
Celery & Celery Beat	Asynchronous task workers and scheduled cron timers
Flask-JWT-Extended	Stateless user session authentication
SQLite	Persistent local relational database store
Chart.js	Analytics visual reporting charts

5. Database Schema & ER Diagram

The database utilizes SQLite, mapping three main tables. Below is the Entity-Relationship (ER) diagram representing the table structure, primary/foreign keys, and relational cardinality:

Table Descriptions:

• **platform_user_accounts:** User account records supporting roles: 'Admin', 'Staff', 'Trekker'. Stores details including contact info, emergency contacts, active status, and blacklisting status.
• **trekking_route_expeditions:** Expeditions tracking names, geographic location, difficulty, pricing, capacity, active slots remaining, guide ID (FK), and status ('Pending', 'Approved', 'Open', 'Closed', 'Completed', 'Cancelled').
• **participant_booking_records:** Slot booking table mapping a Trekker to an expedition. Uses unique transaction reference codes, records participant counts, special notes, and cancellation details. Constrained by a UNIQUE index on (trekker_id, expedition_id).



6. API Resource Endpoints

Module	Method	Endpoint URL	Description
Auth	POST	/api/auth/register	Registers a new Trekker account.
Auth	POST	/api/auth/login	Validates credentials, issues JWT access and refresh tokens.
Auth	POST	/api/auth/refresh-token	Rotates JWT access token.
Auth	GET	/api/auth/me	Retrieves user profile data.
Auth	PATCH	/api/auth/update-profile	Updates user emergency contacts, phone, and bio.

Auth	PATCH	/api/auth/change-password	Secures credential modification.
Auth	DELETE	/api/auth/logout	Revokes JWT sessions.
Admin	GET	/api/admin/dashboard/metrics	Fetches platform visual metrics (counts, charts).
Admin	GET	/api/admin/expeditions	Retrieves all system routes.
Admin	POST	/api/admin/expeditions	Spawns new trekking routes.
Admin	PATCH	/api/admin/expeditions/<id>	Updates routes and guide allocations.
Admin	DELETE	/api/admin/expeditions/<id>	Deletes system routes.
Admin	GET	/api/admin/users	Filters and details system user lists.
Admin	POST	/api/admin/staff	Provisions new staff guides.
Admin	PATCH	/api/admin/users/<id>/blacklist	Temporarily/permanently blacklists user accounts.
Staff	GET	/api/staff/dashboard/summary	Summary data for the logged guide.
Staff	GET	/api/staff/assigned-expeditions	Roster of assigned guide treks.
Staff	PATCH	/api/staff/assigned-expeditions/<id>/operational-update	Adjusts slot availability or toggles status (Started/Completed).
Staff	GET	/api/staff/assigned-expeditions/<id>/participants	View booked participant names.
Trekker	GET	/api/trekker/expeditions/open	Fetches open treks (caching filters).
Trekker	GET	/api/trekker/bookings	Lists own booking history.
Trekker	POST	/api/trekker/bookings	Atomically reserves slots on open treks.
Trekker	PATCH	/api/trekker/bookings/<id>/cancel	Cancels a booking, restoring slots.
Trekker	POST	/api/trekker/bookings/export-csv	Initiates background export CSV delivery.

7. Project Structure & Implemented Features

Directory Structure:

```
trekking-app-v2/
├── backend/
│   ├── app/
│   │   ├── __init__.py          # Flask app initialization and blueprint registration
│   │   ├── config.py            # Environment configurations (dev, test, production)
│   │   ├── extensions.py        # Database, cache, and celery instantiation
│   │   ├── models.py            # SQLAlchemy database models for users, treks, and bookings
│   │   ├── schemas.py           # Marshmallow schemas for validating request data
│   │   ├── auth_routes.py       # User authentication, registration, and profile updates
│   │   ├── admin_routes.py      # Admin controllers for routes, staff creation, and blacklisting
│   │   ├── staff_routes.py      # Staff routes for updating statuses and viewing rosters
│   │   ├── trekker_routes.py    # Trekker routes for searching, booking, and cancelling treks
│   │   └── tasks.py             # Celery background tasks for emails and reports
│   ├── instance/
│   │   └── trekking_platform.db # Main SQLite database file
│   ├── celery_worker.py         # Entry point to run the Celery worker
│   ├── run.py                   # Entry point to run the Flask backend server
│   ├── requirements.txt         # List of Python dependencies
│   └── .env                     # App configurations (secrets, SMTP settings, etc.)
├── frontend/
│   ├── index.html               # Main HTML entry file for the Vue.js app
│   ├── app.js                   # Main JavaScript file containing Vue logic and API calls
│   ├── index.css                # Styling stylesheet (Bootstrap customizations and glassmorphism)
│   ├── manifest.json            # PWA manifest configuration
│   ├── sw.js                    # Service worker for offline caching and fallbacks
│   ├── offline.html             # Offline fallback page
│   └── assets/                  # High-resolution icons and branding logos
├── Project_Report.pdf           # Detailed project report in PDF format
└── README.md                    # Project run guide and overview document
```

Core Implemented Features:

• Multi-Role Portal Architecture: Dedicated dashboards and security rules for Administrator, Staff Guide, and Trekker roles.

• Unified Auth & RBAC: Secure JWT authentication utilizing access tokens and refresh tokens.

• Transactional Slot Integrity: Safe booking transaction logic protecting slots, auto-closing full treks, and auto-reopening cancelled bookings.

• Redis Caching & Invalidation: Public explore routes cached in Redis with active cache invalidation hooks upon status/slot updates.

• Asynchronous background jobs: Daily confirmed trekker email reminders, monthly analytics HTML reporting, and on-demand CSV history export sent to emails.

• PWA Offline Capability: Implements service worker caching with a dedicated offline fallback page.

8. Presentation Video

Video Link: https://drive.google.com/file/d/1ec7bHpU6yqY_Og0DaKeSmBFxTuqvrSu9/view?usp=sharing